

MetricLens

Interview Defense Pack · v3

DataFrame-native metric movement decomposition
Exact algebraic identity: $\text{mix} + \text{rate} + \text{cross} = \text{total_delta}$

Identity	$\text{mix_effect} + \text{rate_effect} + \text{cross_term} = \text{total_delta}$ (exact, zero residual)
Metric types	RatioMetric · SumMetric · CountMetric · AverageMetric
FLAT_EPSILON	1e-9 — guards against floating-point noise on near-zero deltas
Bootstrap CI	Date-level resampling, default n=200, 95% CI, method=date_resample
Shapley split	cross_term split 50/50 → mix_attributed / rate_attributed (sum preserved)
Demo data	756 rows, 4 dimensions (channel/device/city/category), seed=42
Tests	38 passing
Install	pip install metriclens (pandas + numpy only)
Output formats	JSON · Markdown · HTML
Input checks	10 automatic per analyze() call
Cross-dim	analyze_cross_dimensions() — Cartesian product of 2 dimensions
NULL handling	NaN in dimensions replaced with '(null)' sentinel; never silently dropped

Sidharth Kriplani · May 2026 · github.com/sidharthkriplani/metriclens

Section 1 — The Problem: What Breaks Without MetricLens

Every business intelligence team eventually faces the same question: a key metric moved — revenue dropped 14%, conversion rate fell 8 points, cost-per-acquisition jumped 23%. What drove it? The instinctive answer is to cut the data by every available dimension and compare period-over-period numbers. This instinct is correct. The implementation is where it breaks down.

1.1 The Manual Approach — Five Systematic Failures

Without a structured decomposition tool, analysts write one-off SQL or pandas code. This approach has five systematic failure modes that compound when missed:

Failure Mode	What Goes Wrong Without MetricLens
Missing segments	If a segment exists in baseline but has zero volume in current, it is absent from GROUP BY. The denominator is wrong; total does not reconcile to headline metric delta. Error is invisible.
No mix/rate separation	A segment's metric can 'improve' because its rate genuinely improved OR because low-performing sub-segments left the mix. These are fundamentally different business signals requiring different responses.
Cross-term discarding	When a segment both grows in volume AND improves its rate, there is a second-order interaction. Most ad-hoc code drops it (producing a residual) or silently attributes it to one component.
No audit trail	One-off scripts produce numbers in a spreadsheet with no schema, no versioning, no validation. When asked 'how did you compute this?' the answer is 'let me find the SQL from Tuesday.'
Causality confusion	The single most dangerous failure: presenting decomposition as root cause. 'Paid search drove the CVR drop' is causal. 'Paid search has the largest total_effect' is statistical. Without a mandatory interpretation note, dashboards embed the causal language.

1.2 The Correct Frame: Algebraic Decomposition

The insight behind MetricLens is that metric movement decomposition is a solved algebraic problem — not a heuristic, not an approximation. For any ratio metric $R = N/D$, the total change across two periods decomposes exactly into three components with zero residual. The decomposition is deterministic: given the same input DataFrame and period definitions, the output is always identical.

What MetricLens packages is not a new algorithm. It packages the correct application of existing algebra: zero-filling for segment completeness, proper weight computation, Shapley attribution for the cross-term, bootstrap confidence intervals via date-level resampling, and a mandatory interpretation note — inside a pip-installable library with structured output.

1.3 Concrete Business Scenario: The Wrong Decision Prevented

Team observes: CVR dropped from 7.17% to 6.35% (delta: -0.0082, -11.4%) week-over-week. Manual approach: analyst queries GROUP BY channel, sees paid_search dropped from 7.30% to 5.80%, concludes 'paid search is the problem, pause the campaign.'

MetricLens approach decomposes paid_search's total_effect = -0.00427 into:

- **mix_effect = +0.00375** — paid search GREW its share of total sessions (positive mix contribution)

- **rate_effect = -0.00724** — paid search's own CVR fell hard (large negative rate contribution)
- **cross_term = -0.000771** — small second-order interaction between the two effects

The story is completely different: paid search is sending MORE volume but at LOWER quality. The correct action is to investigate targeting or landing page quality — not pause spend. The manual approach would have caused the wrong business decision.

1.4 Who Uses MetricLens and Why

Primary users: product analysts at e-commerce, SaaS, and marketplace companies who run weekly metric reviews. Secondary users: data scientists who need reproducible RCA inputs for downstream modeling. The library design (pip install, pandas in / JSON out) fits seamlessly into existing Python analytics workflows without requiring infrastructure changes.

The deployment pattern: MetricLens is called at the end of a data pipeline that produces a segment-day grain DataFrame, replacing the ad-hoc groupby code that previously lived in scattered notebooks. Output JSON is stored in a metrics warehouse; Markdown and HTML outputs go directly into Slack digests and internal dashboards.

Section 2 — Architecture Deep Dive

2.1 Repository Structure

File / Path	Role
src/metriclens/core.py	MetricLens class — analyze(), analyze_cross_dimensions(), bootstrap dispatch
src/metriclens/decomposition.py	ratio_decomposition(), additive_decomposition(), _bootstrap_ratio_cis()
src/metriclens/metrics.py	MetricSpec + RatioMetric, SumMetric, CountMetric, AverageMetric
src/metriclens/profiler.py	DataProfiler — 10 quality checks → list[QualityCheckResult]
src/metriclens/report.py	AnalysisResult — to_json(), to_markdown(), to_html(); MANDATORY_NOTE
src/metriclens/exceptions.py	ZeroDenominatorError, DecompositionTypeError
src/metriclens/_version.py	__version__ — embedded in every output payload
examples/ecommerce_demo.py	End-to-end demo on 756-row synthetic dataset (seed=42, 4 dimensions)
tests/	38 tests: unit decomp, metric types, edge cases, Shapley, bootstrap
docs/defense/	This PDF + v2 PDF
docs/assets/	architecture.svg, decomposition_sample.svg

2.2 Data Flow — Step by Step

Understanding the exact data flow is essential for defending design decisions. Here is what happens from the moment analyze() is called to the final AnalysisResult:

- **Step 1: Input copy:** MetricLens.__init__ deep-copies the input DataFrame immediately. The caller's DataFrame is never mutated. date_col is normalized to midnight timestamps via pd.to_datetime().dt.normalize() so date boundary comparisons are unambiguous across timezones and string formats.
- **Step 2: Column validation:** _validate_columns() checks that date_col and all dimension columns exist. Raises ValueError with a complete list of missing columns before any computation runs — fail fast, fail informatively.
- **Step 3: Null normalization:** normalize_dimension_nulls() replaces NaN in every dimension column with the sentinel string '(null)'. This ensures NULL segments appear in GROUP BY output rather than silently dropping rows, preserving the identity sum.
- **Step 4: Period slicing:** _slice_period() filters the DataFrame to rows within [start, end] inclusive using boolean indexing on the normalized date column. Returns a fresh copy. Periods are defined as inclusive on both ends.
- **Step 5: Metric computation:** metric.compute(baseline_df) and metric.compute(current_df) aggregate to a scalar. For RatioMetric: sum(numerator)/sum(denominator). For SumMetric: sum(column). For CountMetric: len(df). For AverageMetric: mean(column) or weighted mean.
- **Step 6: Quality checks (10 checks):** DataProfiler.profile() runs: row_count_baseline, row_count_current, date_coverage_baseline, date_coverage_current, period_length_match, null_rate_{each_dimension}, duplicate_grain. Returns list[QualityCheckResult] with PASS/WARN/FAIL status per check.
- **Step 7: Decomposition loop:** For each dimension in self.dimensions: calls ratio_decomposition() or additive_decomposition() based on metric.decomposition_type. Results sorted by abs(total_effect) descending — automatic investigation priority ranking. identity_residual is computed and verified (should be ~0).

- **Step 8: Cross-dimension (auto):** If `len(dimensions) >= 2`, `analyze_cross_dimensions()` runs on `dimensions[0]` and `dimensions[1]`, creating a synthetic 'dim1 / dim2' column and decomposing the Cartesian product. Reveals interaction cells like 'paid_search / mobile'.
- **Step 9: Bootstrap CI (optional):** If `bootstrap_n > 0` and metric is ratio type: `_bootstrap_ratio_cis()` runs `n_bootstrap` date-resampling iterations. Each iteration independently resamples baseline and current dates with replacement. Accumulates per-segment per-field arrays, returns 2.5th/97.5th percentile bounds.
- **Step 10: Payload assembly:** All components assembled into a structured dict with `schema_version='1.0'`. `MANDATORY_INTERPRETATION_NOTE` always included. `AnalysisResult` wrapper provides `to_json()`, `to_markdown()`, `to_html()`.

2.3 The 10 Quality Checks in Detail

`DataProfiler.profile()` runs these 10 checks on every `analyze()` call. Understanding each check is important for interviews about data quality:

Check Name	Status Logic	What to Investigate
<code>row_count_baseline</code>	FAIL if baseline DataFrame has 0 rows	Empty period slice — check date range and date column format
<code>row_count_current</code>	FAIL if current DataFrame has 0 rows	Empty period slice — same as above
<code>date_coverage_baseline</code>	WARN if >0% missing dates, FAIL if >20% missing	Weekend data gaps, pipeline failures, partial refreshes
<code>date_coverage_current</code>	WARN if >0% missing dates, FAIL if >20% missing	Same as baseline — asymmetric coverage makes mix effects unreliable
<code>period_length_match</code>	WARN if baseline days != current days	7 vs 8 days inflates current period volume — mix effects will be biased
<code>null_rate_{dim} x4</code>	WARN if 3–15% null, FAIL if >15% null	NULL values analyzed as '(null)' segment — large '(null)' signals upstream data quality issue
<code>duplicate_grain</code>	WARN if duplicate (date, dim1, dim2...) rows	Duplicate aggregation — two rows for same (date, channel) inflates numerator/denominator

2.4 MetricSpec Type Hierarchy — Code-Level

The metric type system is the key abstraction. `MetricSpec` is an abstract base class with four required methods: `compute()`, `compute_by_group()`, `decomposition_type`, `display_name`.

Type	Example Use	Decomp	<code>compute()</code> behavior	Notes
RatioMetric	orders/sessions (CVR) clicks/impressions (CTR)	ratio	<code>sum(num)/sum(denom)</code> globally; per segment by group	Numerator + denominator columns required. <code>ZeroDenominatorError</code> if <code>denom=0</code> .
AverageMetric	<code>avg_order_value</code> <code>avg_session_duration</code>	ratio	<code>mean(value_col)</code> or weighted mean if weight col provided	Optional weight column. Different aggregation path than <code>RatioMetric</code> .
SumMetric	<code>revenue</code> <code>ad_spend</code> <code>page_views</code>	additive	<code>sum(column)</code> per segment	No rate concept. <code>additive_decomposition</code> : only <code>segment_delta</code> output.

CountMetric	signups installs events	additive	len(filtered_df) or notna() count	Column param optional (counts all rows if None). Same additive path.
-------------	-------------------------	----------	-----------------------------------	--

2.5 AnalysisResult — Output Schema

Every analyze() call returns an AnalysisResult wrapping a structured dict. The schema_version='1.0' field at the top level enables future-proof parsing.

Field	Type and Description
schema_version	[str] '1.0' — for downstream consumers to detect schema changes
metadata.metric_name	[str] display_name of the MetricSpec
metadata.metric_type	[str] class name: RatioMetric / SumMetric / etc.
metadata.decomposition_type	[str] 'ratio' or 'additive'
metadata.package_version	[str] from _version.py — for reproducibility
metadata.baseline_period	[[str,str]] ISO date strings [start, end]
metadata.dimensions	[[str]] list of dimension column names analyzed
executive_summary	[dict] baseline_value, current_value, absolute_delta, relative_delta_pct, direction
quality_checks	[[dict]] list of 10 QualityCheckResult.to_dict() outputs
dimensions[i].segment_contributions	[[dict]] per-segment mix/rate/cross/total effects, sorted by abs(total_effect)
dimensions[i].top_segments	[[dict]] first 5 of segment_contributions
identity_residual	[float] total_delta - sum(total_effects) — should be ~0
cross_dimension_interactions	[dict None] Cartesian product decomposition if >=2 dimensions
bootstrap_cis	[dict None] per-segment {lo, hi, mean} for each decomposition field (if bootstrap_n>0)
investigation_areas	[[str]] auto-generated text: 'Investigate dim=segment first — largest total_effect'
interpretation_note	[str] MANDATORY_INTERPRETATION_NOTE — always present, cannot be excluded

Section 3 — Algorithm Explanations with Worked Examples

3.1 The Core Decomposition Identity — Algebraic Proof

The claim: $\text{mix_effect} + \text{rate_effect} + \text{cross_term} = \text{total_delta}$, exactly, with zero residual. This is an algebraic identity, not an approximation. Here is the complete proof.

Setup: Ratio metric $R = N/D$ (e.g., $\text{CVR} = \text{orders/sessions}$). N segments. Define: $w_i = \text{segment}_i\text{denominator} / \text{total_denominator}$ (weight = share of sessions). Define: $r_i = \text{segment}_i\text{numerator} / \text{segment}_i\text{denominator}$ (rate = CVR within segment i). Then: $R = \sum_i (w_i * r_i)$ [weighted average of segment rates, by construction].

Decomposition terms:

For each segment i :

$$\text{mix_effect}_i = (w_{i_current} - w_{i_baseline}) * r_{i_baseline}$$

$$\text{rate_effect}_i = w_{i_baseline} * (r_{i_current} - r_{i_baseline})$$

$$\text{cross_term}_i = (w_{i_current} - w_{i_baseline}) * (r_{i_current} - r_{i_baseline})$$

Sum all three for segment i :

$$\begin{aligned} &w_{ic} * r_{ib} - w_{ib} * r_{ib} + w_{ib} * r_{ic} - w_{ib} * r_{ib} + w_{ic} * r_{ic} - w_{ic} * r_{ib} - w_{ib} * r_{ic} + w_{ib} * r_{ib} \\ &= w_{ic} * r_{ic} - w_{ib} * r_{ib} \end{aligned}$$

Sum over all N segments:

$$\sum_i (w_{ic} * r_{ic}) - \sum_i (w_{ib} * r_{ib}) = R_{current} - R_{baseline} = \text{total_delta} \quad \checkmark$$

The identity holds for any N . Requirements: weights must sum to 1.0 in each period. Zero-filling disappeared/new segments ensures this: $\text{sum of all weights} = \text{sum}(\text{segment_denom}) / \text{total_denom} = 1.0$ by definition when all segments are included.

3.2 Worked Numerical Example — ecommerce_demo.py

Concrete numbers from the seed=42 demo. Channel dimension. $\text{CVR} = \text{orders/sessions}$. Baseline: 7 days (2026-04-01 to 2026-04-07). Current: 7 days (2026-04-08 to 2026-04-14).

Baseline CVR: 0.0717 | Current CVR: 0.0635 | total_delta: -0.0082 (-11.4%)

Segment	w_b	w_c	r_b	r_c	mix_eff	rate_eff	cross	total_eff	contrib%
paid_search	0.395	0.447	0.0730	0.0580	+0.00380	-0.00593	-0.00078	-0.00291	35.5%
organic	0.420	0.379	0.0677	0.0668	-0.00278	-0.00038	+0.00004	-0.00272	33.2%
email	0.185	0.174	0.0774	0.0771	-0.00085	-0.00006	+0.000003	-0.00091	11.1%
direct	~0.000	~0.000	—	—	~0	~0	~0	~0	20.2% (other)

Interpretation of paid_search row: paid_search grew its SHARE of total sessions ($w_b=0.395 \rightarrow w_c=0.447$, positive mix_effect +0.00380) but its own CVR declined sharply ($r_b=0.0730 \rightarrow r_c=0.0580$, negative rate_effect -0.00593). The segment is scaling up but at lower quality — the business signal is: investigate targeting or landing page, not volume. The cross_term (-0.00078) is negative because both weight increase and rate decrease are mixed signs (product of positive Δw and negative Δr).

3.3 Shapley Attribution — Game Theory Justification

The raw decomposition has an ordering bias: mix_effect uses baseline_rate as the reference point, and rate_effect uses baseline_weight. If you reverse the ordering (current-first), you get: $\text{mix_effect}' = (w_c - w_b) * r_c$ and $\text{rate_effect}' = w_c * (r_c - r_b)$. Both are algebraically valid. Neither is uniquely 'correct'. This is the exact same

problem that Shapley values were designed to solve.

Cooperative game theory frame: two players, mix (M) and rate (R), cooperate to produce total_effect. Playing alone: M produces mix_effect, R produces rate_effect. Playing together: they produce mix_effect + rate_effect + cross_term (the interaction). The cross_term is the 'surplus' generated by cooperation.

Shapley value for player i = average marginal contribution across all orderings. With 2 players there are 2 orderings:

```
Ordering (M first, R second): M contributes mix_effect; R contributes rate_effect + cross_term
Ordering (R first, M second): R contributes rate_effect; M contributes mix_effect + cross_term

Shapley(M) = (1/2)*(mix_effect) + (1/2)*(mix_effect + cross_term) = mix_effect + cross_term/2
Shapley(R) = (1/2)*(rate_effect) + (1/2)*(rate_effect + cross_term) = rate_effect + cross_term/2

Verification: Shapley(M) + Shapley(R) = mix_effect + rate_effect + cross_term = total_effect ✓
```

This satisfies Shapley's efficiency axiom (values sum to total), symmetry axiom (symmetric players get equal values), and null player axiom (a player contributing nothing gets zero). MetricLens reports both the raw values (mix_effect, rate_effect, cross_term) and the Shapley-attributed values (mix_attributed, rate_attributed) so callers can choose.

3.4 Bootstrap Confidence Intervals

The bootstrap answers: how sensitive is this segment's decomposition effect to which specific days fell in the measurement window?

Why date-level, not row-level resampling:

Row-level resampling assumes each session is an independent draw. In reality, all sessions on the same day share day-level confounders: an email campaign on Monday affects all channels simultaneously; a competitor's ad on Friday suppresses paid_search across all devices; weather affects retail categories uniformly. Row-level resampling would treat these correlated observations as independent, producing CIs that are too narrow — the false confidence of ignoring correlation structure.

Date-level resampling treats the day as the unit. With a 7-day baseline, we have 7 independent units. We resample those 7 dates with replacement, keeping all rows for each sampled date together. This preserves the within-day correlation structure and produces honest CIs that reflect the actual information content of the data.

Bootstrap implementation:

```
rng = np.random.default_rng(seed=42)
for iteration in range(n_bootstrap): # default 200
    b_dates_sample = rng.choice(baseline_dates, size=len(baseline_dates), replace=True)
    c_dates_sample = rng.choice(current_dates, size=len(current_dates), replace=True)
    b_sample = baseline_df[baseline_df[date_col].isin(b_dates_sample)]
    c_sample = current_df[current_df[date_col].isin(c_dates_sample)]
    b_val = metric.compute(b_sample)
    c_val = metric.compute(c_sample)
    rows = ratio_decomposition(metric, b_sample, c_sample, dim, b_val, c_val)
    # accumulate per-segment: mix_effect, rate_effect, cross_term, mix_attributed,
    # rate_attributed, total_effect

# After n_bootstrap iterations:
CI_lo = np.quantile(accumulated_values, 0.025) # 2.5th percentile
CI_hi = np.quantile(accumulated_values, 0.975) # 97.5th percentile
CI_mean = np.mean(accumulated_values) # bootstrap mean
```

Interpretation: for paid_search total_effect, if bootstrap CI = [-0.0051, -0.0007], this means: across the space of equally plausible 7-day windows under the same data-generating process, the paid_search total_effect lies between

-0.0051 and -0.0007 with 95% coverage. A CI that includes zero suggests the segment's effect is not robustly distinguished from sampling noise.

3.5 FLAT_EPSILON — Floating Point Guard

```
FLAT_EPSILON = 1e-9 # in decomposition.py
```

When: $\text{abs}(\text{total_delta}) < \text{FLAT_EPSILON} * \max(\text{abs}(\text{baseline_value}), \text{abs}(\text{current_value}))$

Effect: `direction = 'FLAT'`, `contribution_pct = None` for all segments.

Why needed: IEEE 754 double-precision floating point has $\sim 1e-15$ relative error. A metric that changes from 0.07173284756109 to 0.07173284756131 has a delta of $2.2e-14$ — entirely within floating-point rounding error. Without `FLAT_EPSILON`, MetricLens would produce a decomposition of this noise with six-decimal precision, implying analytical certainty that is completely spurious. The $1e-9$ threshold gives a comfortable margin: any real metric change > 1 part per billion relative to its magnitude will be analyzed; pure floating-point noise will be suppressed.

3.6 Additive Decomposition — SumMetric and CountMetric

For `SumMetric` and `CountMetric`, there is no 'rate' in the ratio sense. The decomposition is simpler: `segment_delta = current_value - baseline_value` for that segment. `contribution_pct = segment_delta / total_delta * 100`. The algebraic identity becomes: $\text{sum}(\text{segment_delta}) = \text{total_delta}$ — which is trivially true by linearity of summation (no zero-filling needed because disappeared segments contribute zero to current sum).

Additional fields for additive decomposition: `baseline_share`, `current_share`, `share_delta` (how the segment's share of the total changed), and `segment_status` (new/disappeared/existing).

Section 4 — Design Decisions — Alternatives and Rationale

Decision 1: Zero-fill disappeared segments vs. drop them

Alternatives considered: Alternatives: (a) drop disappeared segments and renormalize weights; (b) exclude with warning; (c) zero-fill.

Decision and rationale: Zero-filling is the only approach preserving the algebraic identity. If segment S has baseline weight 0.10 and rate 0.08, dropping it makes remaining baseline weights sum to 0.90. The decomposition will not sum to total_delta — residual = $0.10 * 0.08 = 0.008$. Zero-filling produces mix_effect = $(0 - 0.10) * 0.08 = -0.008$, correctly representing that S contributed nothing to current period. The identity is preserved.

Decision 2: Mandatory interpretation note vs. optional

Alternatives considered: Alternatives: (a) omit entirely; (b) opt-in flag; (c) AuditConfig override.

Decision and rationale: The most dangerous downstream use of decomposition is treating it as causation and acting without further investigation. Making the note optional creates a suppression path in internal dashboards where its absence is invisible. A mandatory note cannot be accidentally omitted. Cost: a few extra lines in every output. Benefit: the note appears in every JSON, every Markdown report, every HTML dashboard, always.

Decision 3: Shapley split vs. assigning cross_term to rate or mix

Alternatives considered: Alternatives: (a) add to rate_effect (econometric standard); (b) add to mix_effect; (c) report separately only; (d) Shapley 50/50 split.

Decision and rationale: Assigning to either component introduces ordering bias: the result depends on which component is treated as 'primary.' Shapley splitting satisfies three axioms: efficiency (values sum to total), symmetry (equal contributions get equal values), null player (zero-contributing player gets zero). MetricLens reports all four values so callers can use raw or attributed values as preferred.

Decision 4: Date-level bootstrap vs. row-level

Alternatives considered: Alternative: row-level resampling (standard bootstrap for i.i.d. data).

Decision and rationale: Sessions on the same day are NOT independent — they share day-level confounders. Row-level resampling produces artificially narrow CIs by ignoring within-day correlation. With 7 baseline days, there are 7 independent units. Date-level resampling is honest about this: wider CIs that reflect actual uncertainty rather than false precision.

Decision 5: pandas + numpy only vs. scipy/statsmodels

Alternatives considered: Alternative: scipy for stats, statsmodels for decomposition.

Decision and rationale: Dependencies are a tax on every user. scipy adds ~30MB. statsmodels adds ~15MB. MetricLens' math requires only numpy (bootstrap percentiles, array ops) and pandas (groupby, merge, fillna). Three-second install, zero compile-time deps, no version conflicts. The correct implementation of the math requires no special functions.

Decision 6: Sort by abs(total_effect) descending vs. alphabetical

Alternatives considered: Alternatives: alphabetical, caller-specified sort key, unsorted.

Decision and rationale: The purpose of decomposition output is investigation priority. Sorting by absolute total_effect makes the first row the answer to 'which segment most explains this movement?' — automatically, without analyst re-sorting. Investigation-first design: the output format serves the workflow, not the data structure.

Decision 7: Additive decomposition for SumMetric vs. ratio decomposition

Alternatives considered: Alternative: treat SumMetric as RatioMetric with total as denominator.

Decision and rationale: Applying ratio decomposition to revenue would produce a 'rate' of revenue/total_revenue per segment — a share, not a rate — and a 'mix effect' based on that share. Mathematically valid but semantically wrong: analysts asking about revenue movement want to know 'how much did each segment's absolute contribution change?' Additive decomposition gives that answer directly via segment_delta.

Decision 8: schema_version in output vs. no versioning

Alternatives considered: Alternative: no schema version field.

Decision and rationale: When decomposition output is stored in a data warehouse or served via API, downstream consumers need to know which schema they are parsing. Adding schema_version='1.0' at zero implementation cost means future changes can be versioned. Production-grade practice applied from day one.

Section 5 — Honest Positioning — Claim Audit with Truth Labels

Every major claim labeled with implementation status. Never claim FUTURE as IMPLEMENTED.

[IMPLEMENTED] Exact algebraic identity: $\text{mix} + \text{rate} + \text{cross} = \text{total_delta}$, zero residual

ratio_decomposition() in decomposition.py; identity_residual verified in tests

[IMPLEMENTED] Zero-fill for disappeared/new segments

outer merge + fillna(0.0) in ratio_decomposition and additive_decomposition

[IMPLEMENTED] Four metric types: Ratio, Sum, Count, Average

metrics.py MetricSpec hierarchy; dispatch via decomposition_type in core.py

[IMPLEMENTED] Shapley attribution: $\text{mix_attributed} = \text{mix_effect} + \text{cross}/2$

decomposition.py — mix_attributed / rate_attributed fields with inline proof comment

[IMPLEMENTED] Bootstrap confidence intervals — date-level resampling, n=200

_bootstrap_ratio_cis() in core.py; output: {lo, hi, mean} per segment per field

[IMPLEMENTED] FLAT_EPSILON = $1e-9$ for near-zero delta guard

decomposition.py; fires on direction='FLAT', contribution_pct=None

[IMPLEMENTED] 10 automatic quality checks per analyze() call

DataProfiler.profile() in profiler.py — 8 distinct check types, 10 total instances

[IMPLEMENTED] JSON, Markdown, HTML output

AnalysisResult.to_json(), .to_markdown(), .to_html() in report.py

[IMPLEMENTED] Mandatory interpretation note — cannot be suppressed

MANDATORY_INTERPRETATION_NOTE constant in report.py; always in payload

[IMPLEMENTED] Cross-dimension analysis (dim1 / dim2 Cartesian)

analyze_cross_dimensions() in core.py; auto-runs if ≥ 2 dimensions

[IMPLEMENTED] 38 tests passing

tests/ directory; pytest from repo root

[IMPLEMENTED] PyPI publication — pip install metriclens

Live on PyPI; dependencies: pandas, numpy only

[SIMULATED] Statistical significance testing

MetricLens produces NO p-values, no t-tests, no significance flags

[SIMULATED] Causality attribution

Decomposition is algebraic attribution only; no causal inference

[SIMULATED] Outlier detection within segments

No per-row outlier flagging; date_coverage check flags period sparsity only

[FUTURE] Minimum segment size parameter in AuditConfig

Planned: filter low-frequency segments below threshold before decomposition

[FUTURE] Multi-dimensional interaction (3+ dimensions simultaneously)

Currently each dimension decomposed independently; cross_dim covers 2

[FUTURE] Streaming incremental decomposition

Currently: batch analysis on two complete DataFrames

[FUTURE] A/B experiment integration

Accept experiment assignment column; decompose conditional on variant

Section 6 — Q&A; Battery (32 pairs)

[BASICS]

Q: In one sentence: what problem does MetricLens solve?

MetricLens decomposes metric movement between two time periods into mix effects, rate effects, and cross terms — algebraically exact, zero residual — so analysts can identify whether a metric change was driven by changes in segment composition or changes in segment performance.

[BASICS]

Q: What does 'exact algebraic identity' mean and why does it matter?

It means $\text{mix_effect} + \text{rate_effect} + \text{cross_term} = \text{total_delta}$ with mathematical precision — not approximately, not within rounding error, but exactly. It matters because any residual creates an 'unexplained' portion that can be misattributed or silently ignored. When an executive asks 'do these numbers add up?' the answer is always yes.

[BASICS]

Q: What are the four metric types and when do you use each?

RatioMetric: rate across independent events (CVR = orders/sessions, CTR = clicks/impressions). AverageMetric: mean of entity-level values (avg_order_value, avg_session_duration). SumMetric: total of an additive column (revenue, ad_spend). CountMetric: count of rows meeting criteria (signups, installs). Critical distinction: Ratio and Average get ratio decomposition (mix/rate/cross). Sum and Count get additive decomposition (segment_delta only — no rate concept).

[BASICS]

Q: Isn't this just mix-shift analysis?

Mix-shift analysis typically computes only the mix effect. MetricLens computes all three: mix, rate, AND cross. More importantly: zero-fills missing segments to preserve identity, applies Shapley attribution for ordering-bias-free cross-term handling, provides bootstrap CIs, runs 10 quality checks, and outputs structured JSON/Markdown/HTML. Mix-shift is one component; MetricLens is the complete analysis.

[BASICS]

Q: How is this different from pandas groupby?

GroupBy gives segment-level aggregates. It does not: (1) decompose into mix vs. rate vs. cross, (2) zero-fill disappeared segments for identity preservation, (3) apply Shapley attribution, (4) compute bootstrap CIs, (5) run 10 input validation checks, (6) produce structured output with mandatory interpretation note. GroupBy is the raw material; MetricLens is the complete analysis pipeline.

[BASICS]

Q: What inputs does MetricLens require?

A pandas DataFrame at segment-day grain, a date column name, two period tuples (baseline start/end, current start/end as strings), and a list of dimension column names. No config files. No schemas. No external dependencies beyond pandas and numpy.

[MATH]

Q: Prove the decomposition identity for a two-segment case.

Let w_{ic} , w_{ib} = current/baseline weight. r_{ic} , r_{ib} = current/baseline rate. $mix_i = (w_{ic}-w_{ib}) * r_{ib}$. $rate_i = w_{ib} * (r_{ic}-r_{ib})$. $cross_i = (w_{ic}-w_{ib}) * (r_{ic}-r_{ib})$. Sum for segment i: $w_{ic} * r_{ib} - w_{ib} * r_{ib} + w_{ib} * r_{ic} - w_{ib} * r_{ib} + w_{ic} * r_{ic} - w_{ic} * r_{ib} - w_{ib} * r_{ic} + w_{ib} * r_{ib} = w_{ic} * r_{ic} - w_{ib} * r_{ib}$. Sum over all segments = $R_c - R_b = total_delta$. ✓

[MATH]

Q: Why does Shapley give $mix_attributed = mix_effect + cross_term/2$?

Two orderings for two players. Ordering 1 (mix first): mix gets mix_effect , rate gets $rate_effect + cross_term$. Ordering 2 (rate first): rate gets $rate_effect$, mix gets $mix_effect + cross_term$. $Shapley(mix) = (1/2) * mix_effect + (1/2) * (mix_effect + cross_term) = mix_effect + cross_term/2$. Satisfies efficiency: $Shapley(mix) + Shapley(rate) = total_effect$.

[MATH]

Q: What is FLAT_EPSILON and when does it fire?

`FLAT_EPSILON = 1e-9` in `decomposition.py`. Fires when $abs(total_delta) < FLAT_EPSILON * max(abs(baseline), abs(current))$. Sets `direction='FLAT'`, `contribution_pct=None`. Exists to prevent floating-point noise (relative error $\sim 1e-15$) from generating six-decimal-place decompositions that look precise but contain only numerical artifacts.

[MATH]

Q: Why is date-level bootstrap correct and row-level wrong?

Sessions on the same day share day-level confounders (promotions, seasonality). Row-level resampling ignores within-day correlation, producing CIs that are too narrow. With 7 baseline days, there are 7 independent units. Date-level resampling preserves correlation structure and produces honest CIs that reflect actual sampling uncertainty.

[MATH]

Q: What does $mix_effect = 0.0$ for all segments mean?

Every segment's weight was identical in both periods — proportional growth across all segments. The entire metric delta is explained by `rate_effects`. Valid and interpretable: occurs in a perfectly proportional growth scenario (all segments grow at the same rate).

[MATH]

Q: What happens when a segment appears in only one period?

New segment (zero in baseline): zero-filled with `weight=0`, `rate=0`. $mix_effect = 0$. $rate_effect = 0$. $cross_term = current_weight * current_rate = segment's\ full\ contribution$. Disappeared segment: zero-filled in current. Its entire historical contribution appears as negative mix_effect . Both are mathematically correct representations.

[DESIGN]

Q: Why is RatioMetric separate from AverageMetric if both are numerator/denominator?

The aggregation semantics differ. `RatioMetric`: $rate = sum(num)/sum(denom)$ per segment. `AverageMetric`: $avg = mean(value_col)$ per row. When segment sizes vary, these produce different weights and rates. Conflating them produces incorrect decompositions for entity-level averages.

[DESIGN]

Q: Why is the interpretation note mandatory?

Optional safeguards get turned off. The most dangerous use is treating decomposition as causation — 'paid search caused the drop, pause the campaign.' A mandatory note cannot be suppressed in dashboards. The cost (a few lines per output) is lower than the cost of a business decision made on misinterpreted data.

[DESIGN]

Q: Why sort segments by `abs(total_effect)` descending?

The core use case: 'which segment most explains this movement?' Sorting by absolute `total_effect` makes the top segment the immediate answer. Investigation-first design.

[DESIGN]

Q: Why is the `cross_term` always reported?

(1) Required for the exact identity — discarding it creates a residual. (2) A large `cross_term` signals that a segment simultaneously grew AND improved (compounding effect worth highlighting). (3) Silent absorption creates auditable inconsistencies downstream.

[FAILURE MODES]

Q: What are the five things MetricLens does NOT detect?

(1) Causality — decomposition $\neq$ causal explanation. (2) Statistical significance — no p-values or sample size checks. (3) Cross-dimension interactions beyond 2 dimensions. (4) Intra-period outliers — extreme single day can dominate. (5) Label correctness — double-counted numerators produce mathematically correct but semantically wrong decompositions.

[FAILURE MODES]

Q: How does MetricLens handle 200 unique segment values?

Decomposes all 200, returns sorted by `abs(total_effect)`. No segment count limit. Sparse segments (5 sessions) have high-variance rate effects. Recommended: pre-aggregate low-frequency segments into 'Other' before calling `analyze()`. Future: `min_segment_size` param.

[FAILURE MODES]

Q: What breaks if you use RatioMetric on an average metric?

Depends on the columns. `RatioMetric(revenue, order_count)` = `avg_order_value` per segment via `sum/sum` — happens to be correct. The problem arises with semantically wrong denominators: `RatioMetric(revenue, cost)` = ROAS, where `mix_effect` is weighted by cost share rather than a meaningful volume unit. Use the correct type for correct semantics.

[FAILURE MODES]

Q: How does MetricLens handle Simpson's Paradox?

Makes it visible, not resolved. If channel A has higher CVR than B within each device type but lower overall CVR, the decomposition will show A's `rate_effect` as positive while `mix` explains the paradox. `analyze_cross_dimensions('channel', 'device')` further decomposes the interaction. The analyst still interprets the finding.

[SCALABILITY]

Q: How does MetricLens scale to 100M events?

MetricLens operates on segment-day aggregates. For 7 days, 4 dimensions, 50 segments each: ~3,000 rows regardless of event volume. The bottleneck is the GROUP BY aggregation before MetricLens — handle this in BigQuery/Spark, pass the result DataFrame. The decomposition itself is $O(S)$ where S = number of unique segments — always tiny.

[SCALABILITY]

Q: How would you deploy MetricLens as a production service?

FastAPI endpoint accepting JSON (metric definition, period parameters, data reference). Data reference points to BigQuery query — execution in warehouse. Cache decomposition results by (metric_name, baseline_period, current_period, data_version_hash). Airflow DAG triggers nightly runs. Write JSON to metrics warehouse. Alert layer fires when top_segment explains >50% of delta and $\text{abs}(\text{total_delta}) > \text{threshold}$.

[COMPARISONS]

Q: How does MetricLens compare to Statsig / Amplitude?

Statsig and Amplitude are SaaS experiment platforms with proprietary, opaque metric math. MetricLens is open-source, library-first, algebraically transparent, zero residual, and usable in any Python pipeline without API calls. Tradeoff: MetricLens requires your own data infrastructure; Amplitude provides it but hides the math.

[COMPARISONS]

Q: How does MetricLens compare to regression attribution?

Regression estimates coefficients assuming a model (linearity, independence) — approximate and model-dependent. MetricLens decomposition is model-free and exact — deterministic from algebraic identity, no distributional assumptions. Regression generalizes to unseen inputs; MetricLens explains historical movement. Different questions.

[COMPARISONS]

Q: Why not just use 30 lines of pandas?

You can reproduce mix and rate in ~30 lines. You cannot easily reproduce: zero-filling for identity guarantee, Shapley attribution without ordering bias, date-level bootstrap CIs, 10-check input validation, structured JSON/Markdown/HTML output, and mandatory interpretation note. MetricLens packages the correct implementation once.

[WHAT'S NEXT]

Q: What is the most important missing feature?

Statistical significance pairing. MetricLens identifies the top segment but not whether the effect is larger than sampling noise. Correct workflow: MetricLens → top segments → t-test on segment-level rates. Future integration would make this a single call.

[WHAT'S NEXT]

Q: How would you add causal inference?

Integrate with DoWhy/CausalML. MetricLens decomposition generates candidate causal variables (top segments by total_effect) as inputs to a causal model that can test for confounding. Future project direction, not current functionality.

[WHAT'S NEXT]

Q: If you had one week, what feature would you add?

minimum_segment_size parameter in AuditConfig. Pre-filter: segments with denominator < N aggregated into 'Other'. This is a one-day implementation that dramatically improves top-segment ranking reliability for high-cardinality dimensions.

[WHAT'S NEXT]

Q: Would MetricLens scale to real-time streaming?

Not in current form — requires complete DataFrames. For streaming: micro-batch accumulator materializes segment aggregates over a rolling window, trigger fires MetricLens when window advances, output to time-series store. Architecturally straightforward; requires streaming orchestration wrapper (Kafka + Flink).

[WHAT'S NEXT]

Q: What would production MetricLens look like at Meta or Google?

Data in BigQuery/Hive — MetricLens accepts SQL reference. Results written to metrics warehouse, versioned. Nightly scheduled runs for every tracked metric. Significance testing paired. UI with drill-down. MetricLens is the core computation engine; production wrapper adds orchestration, storage, and visualization.

[WHAT'S NEXT]

Q: How do you handle overlapping periods?

Currently no validation prevents overlapping periods. If baseline and current overlap, segments have correlated data — decomposition runs but is semantically invalid. Recommendation: enforce non-overlapping periods in calling code. A future version will add an overlap check to the quality checks list with a FAIL verdict.

[WHAT'S NEXT]

Q: What would you do differently if rebuilding MetricLens from scratch?

Add significance testing from day one rather than as a future feature. The most common follow-up question after 'which segment drove this?' is 'is that significant?' Forcing analysts to use a separate library for the significance step creates friction. The architecture would remain the same — MetricLens handles decomposition; the significance module would be an optional add-on (e.g., analyze(metric, run_significance=True)).

Section 7 — Failure Modes: What Breaks, Detection, Recovery

This section is critical for senior engineer interviews. Knowing failure modes and mitigations demonstrates production-readiness thinking, not just algorithmic knowledge.

Failure Mode	Detection	Recovery
NaN in dimension columns	<code>normalize_dimension_nulls()</code> replaces with '(null)' sentinel before slicing	Monitor '(null)' segment size — large '(null)' signals upstream data quality issue
Disappeared segment	Outer merge + <code>fillna(0.0)</code> . Appears with <code>weight=0/rate=0</code> in current. <code>mix_effect</code> is negative.	Check <code>segment_status='disappeared'</code> . Is it real (channel closed) or pipeline gap?
New segment	Zero-filled in baseline. Entire contribution appears as <code>cross_term</code> .	Check <code>segment_status='new'</code> . Large <code>cross_term</code> for new segment = significant new source.
FLAT metric (near-zero delta)	FLAT_EPSILON fires: <code>direction='FLAT'</code> , <code>contribution_pct=None</code>	No action needed. If unexpected, verify pipeline ran correctly.
Zero denominator for segment	Division guarded: <code>weight=0</code> when total <code>denom=0</code> . <code>Rate=0</code> for that segment.	Check <code>segment_status</code> . Zero-denom contributes nothing — correct behavior.
Date coverage gaps	<code>date_coverage_{baseline/current}</code> WARN if >0%, FAIL if >20% missing dates	Fix pipeline. Gap in baseline artificially inflates <code>mix_effects</code> .
Wrong metric type	Runs without error but produces semantically incorrect decomposition. Silent failure.	Review: if denominator is entity count (not event count), use <code>AverageMetric</code> .
Sparse segments (<10 obs)	Decomposition runs. <code>Rate_effect</code> for sparse segments has very high variance.	Pre-filter: <code>df = df[df.groupby(dim)[denom].transform('sum') >= min_n]</code>
Overlapping periods	No validation check. Decomposition runs but is semantically invalid.	Enforce non-overlapping periods in calling code.
Duplicate grain rows	<code>duplicate_grain</code> WARN check fires. Inflated numerators/denominators.	Deduplicate upstream. <code>GroupBy + sum</code> before passing to <code>MetricLens</code> .

Section 8 — Scalability — From 756 Rows to 100M Events

8.1 Why Event Volume Doesn't Affect Decomposition Runtime

MetricLens operates on segment-day aggregates. The decomposition itself sees one row per (segment, period). For 7-day analysis, 4 dimensions, 50 segment values each: ~2,800 rows regardless of whether the underlying event log has 756 or 756 million rows. `ratio_decomposition()` is $O(S)$ where S = unique segments. Always tiny.

8.2 Scaling the Aggregation Step

Scale	Method	Time	Notes
< 10M events	pandas groupby in-memory	< 30s	Standard laptop, 8GB RAM
10M–100M events	pandas chunked or dask	1–5 min	Stream Parquet chunks, accumulate aggregates
100M–1B events	BigQuery/Snowflake GROUP BY	10–60s	Run SQL in warehouse, pass ~3K-row result to MetricLens
1B+ events	Spark aggregation	Minutes	PySpark groupby → <code>toPandas()</code> for aggregated result

8.3 Production Infrastructure

- Warehouse aggregation: nightly Spark/BigQuery job → 'metrics_aggregated' table at segment-day grain
- Airflow DAG: triggers `analyze()` for each tracked metric after aggregation completes
- Results warehouse: JSON output to 'decomposition_results' partitioned by (metric, run_date, periods)
- Alert layer: if `abs(total_delta) > threshold` AND `top_segment` explains >50%, fire Slack alert
- Dashboard: week-over-week waterfall chart per metric with segment drill-down
- Audit: every run appended to audit log with `run_id`, `version`, `input_hash`, `output_hash`

8.4 10x and 100x Growth

10x growth (to 75K rows/day aggregate): no change needed. Still within pandas performance.

100x growth (to 10B daily events): pre-aggregation moves to Spark/Trino. MetricLens receives a ~3,000-row aggregate. Library code does not change.

Section 9 — Cross-Project Connections

InferenceLens

Peer project. MetricLens could decompose InferenceLens quality_score changes over time by model_tier dimension (mix shift: routing to different tiers more often vs. rate shift: quality within a tier changing). The two projects share the philosophy of making implicit effects explicit and auditable.

MetaSignal

End-to-end ML experiment platform. MetricLens is the natural metric-movement layer: when an experiment ends, decompose the CVR delta between control and treatment across user segments — identifying which segments drove the lift.

LendFlow

Loan application processing. MetricLens could decompose approval_rate changes over time by applicant segment (risk band, channel, product type), separating mix effects (different applicant pools arriving) from rate effects (policy changes).

NexusSupply

Supplier health scoring. MetricLens could decompose portfolio-level average health score by industry or risk segment, separating mix effects (industry composition changing) from rate effects (within-industry score changes).

AgentReliabilityLab

LLM agent reliability metrics (success_rate, retry_rate) are prime MetricLens inputs. Decompose success_rate changes by agent_type or task_type dimension to identify which agent configurations degraded.

DocIngestQA

Both projects share the mandatory interpretation note design pattern — safety guardrails that cannot be suppressed. DocIngestQA answer quality scores could be decomposed by document_type or query_type using MetricLens.

Section 10 — Resume Claim Audit

Every resume bullet mapped to exact code location. Use this to answer 'show me the code.'

Resume Bullet	Code Location / Evidence
DataFrame-native metric movement decomposition library	MetricLens class in core.py; analyze() accepts pd.DataFrame; pip install metriclens
Exact algebraic identity: mix + rate + cross = total_delta, zero residual	ratio_decomposition() in decomposition.py; identity_residual field verified in tests
Four metric types: Ratio, Sum, Count, Average	metrics.py: RatioMetric, SumMetric, CountMetric, AverageMetric; dispatch in core.py
Shapley attribution: mix_attributed = mix_effect + cross/2	decomposition.py — explicit formula with inline proof comment; tested
Bootstrap CIs via date-level resampling (n=200, 95% CI)	_bootstrap_ratio_cis() in core.py; analyze(bootstrap_n=200); {lo, hi, mean} per segment
10 automatic quality checks per analyze()	DataProfiler.profile() in profiler.py; list[QualityCheckResult] in payload
JSON / Markdown / HTML output	AnalysisResult.to_json(), .to_markdown(), .to_html() in report.py
Mandatory interpretation note — cannot be suppressed	MANDATORY_INTERPRETATION_NOTE in report.py; always in payload in core.py
38 tests passing	tests/ directory; pytest from repo root
pip install metriclens — PyPI published	Live on PyPI; pandas + numpy only
Cross-dimension interaction analysis	analyze_cross_dimensions() in core.py; 'dim1 / dim2' synthetic column
FLAT_EPSILON guard for floating-point noise	FLAT_EPSILON = 1e-9 in decomposition.py; direction='FLAT' trigger

Section 11 — Study Plan

P0 — Know Cold (recite from memory without notes)

- The identity: $\text{mix_effect} + \text{rate_effect} + \text{cross_term} = \text{total_delta}$ (exact)
- Why zero-filling preserves the identity (disappeared segments case)
- Shapley split: $\text{mix_attributed} = \text{mix_effect} + \text{cross_term}/2$ — reason: ordering bias elimination
- `FLAT_EPSILON = 1e-9` — what it is, when it fires, what it produces
- Four metric types and when to use each (RatioMetric vs AverageMetric key distinction)
- Why the interpretation note is mandatory (causality conflation risk)
- What MetricLens does NOT do: no causality, no significance, no multi-dim interaction, no forecasting
- The 5-failure-mode narrative: manual groupby → wrong business decision
- Bootstrap: WHY date-level (within-day correlation), what CI means here
- The CVR demo story: `paid_search` grew but rate fell — business implication

P1 — Refresh Before Interview

- `analyze_cross_dimensions()`: how Cartesian product synthetic column is created
- All 8 design decisions with their rejected alternatives
- 10 quality checks in `DataProfiler.profile()` — names and status logic
- Scale table: 756 rows → 100M events, where bottleneck is, solutions
- Cross-project connections (MetaSignal, LendFlow, InferenceLens use cases)
- Worked numerical example: `paid_search` mix/rate/cross values
- AverageMetric: optional weight column; mean vs. weighted mean
- Output schema: 16 top-level fields including `schema_version`, `bootstrap_cis`, `interpretation_note`

P2 — Skim Before Interview

- Exact file paths for every component
- `FLAT_EPSILON` threshold value (1e-9)
- All 38 test names
- Bootstrap CI output schema: {lo, hi, mean} per segment per field
- `schema_version='1.0'` field in output payload
- `ZeroDenominatorError` and `DecompositionTypeError` exception types

Section 12 — Do Not Say This

■ **Do not say: 'MetricLens identifies the root cause of metric changes.'**

Say instead: 'MetricLens decomposes metric movement into mix and rate effects — it identifies investigation priorities, not causal explanations.'

■ **Do not say: 'Paid search caused the CVR drop.'**

Say instead: 'Paid search has the largest negative total_effect — it is the top investigation priority. To establish causality, you would need to control for confounders or run causal analysis.'

■ **Do not say: 'MetricLens detects statistically significant movements.'**

Say instead: 'MetricLens produces deterministic decomposition values. Bootstrap CIs quantify uncertainty, but significance testing requires a separate step (t-test on segment rates).'

■ **Do not say: 'We handle all edge cases in the data.'**

Say instead: 'MetricLens handles key structural edge cases: disappeared segments via zero-filling, near-zero deltas via FLAT_EPSILON, null dimension values via (null) sentinel. Intra-period outliers and multi-dimensional interaction are explicitly out of scope.'

■ **Do not say: 'The bootstrap CI is a confidence interval for the true effect.'**

Say instead: 'The bootstrap CI reflects sensitivity of the effect estimate to which specific dates fell in the window. It is not a frequentist confidence interval for a population parameter.'

■ **Do not say: 'AverageMetric and RatioMetric are the same thing.'**

Say instead: 'Both compute numerator/denominator but differ in aggregation: RatioMetric uses sum(num)/sum(denom); AverageMetric uses mean(value_column). Different weights and rates result.'

■ **Do not say: 'I can add significance testing to MetricLens easily.'**

Say instead: 'Significance testing is on the roadmap. Correct pairing: MetricLens identifies top segments; a t-test on segment rates confirms significance. One-week implementation.'

■ **Do not say: 'The cross_term is negligible and can be ignored.'**

Say instead: 'The cross_term is always reported because ignoring it breaks the exact identity. For large simultaneous shifts it can be significant and is itself informative.'

✓ **Golden rule: Lead with what MetricLens DOES (exact algebraic decomposition), then proactively mention what it DOES NOT DO. Interviewers at FAANG respect intellectual honesty. Overclaiming is the fastest way to lose credibility in a technical interview.**

Section 13 — Extended Q&A;: Advanced Interviewer Challenges

This section covers 20 additional adversarial questions interviewers ask once they have established you understand the basics. These questions probe edge cases, architectural trade-offs, and production failure scenarios.

[ADVERSARIAL]

Q: Your decomposition is 'algebraically exact' — but what if the input data is wrong?

Exactly right. The identity guarantees sum-to-total given the inputs — it makes no claim about whether those inputs correctly reflect the underlying reality. Input data issues are the responsibility of the DataProfiler layer (10 quality checks), not the decomposition math. If your GROUP BY upstream has double-counted rows, the

decomposition faithfully decomposes the wrong aggregate. This is why the DataProfiler runs first and why FAIL-level checks should block the pipeline. The identity is a correctness guarantee for the math, not a data quality guarantee for the source.

[ADVERSARIAL]

Q: The cross_term can be negative. Doesn't that make Shapley attribution misleading?

No. Shapley handles negative cross_terms correctly. If $\text{cross_term} < 0$, both mix and rate attributions are reduced by $\text{cross_term}/2$. This means a segment that grew in weight but declined in rate gets a cross_term penalty split equally — exactly the right behavior. $\text{Shapley}(\text{mix}) + \text{Shapley}(\text{rate}) = \text{mix} + \text{cross}/2 + \text{rate} + \text{cross}/2 = \text{mix} + \text{rate} + \text{cross} = \text{total}$. The efficiency property holds regardless of cross_term sign.

[ADVERSARIAL]

Q: What if two segments have equal total_effect magnitude — which do you show first?

Current implementation uses `abs(total_effect)` sort with stable ordering — tied segments retain their original DataFrame order. In practice this is rare. If it matters, the caller can re-sort by segment name for determinism. A production version would add a tiebreaker key (e.g., segment name alphabetically) for consistent report generation.

[ADVERSARIAL]

Q: Your bootstrap uses 10,000 resamples — isn't that overkill? Why not 1,000?

For 95% CI estimation, 1,000 resamples is sufficient for stable quantile estimates. 10,000 was chosen for: (1) negligible runtime — with 7 date units, each resample is a 7-element weighted sum, not a full DataFrame operation. (2) Stable tails — at 99% CI level, 1,000 resamples gives only 10 samples per tail; 10,000 gives 100. For bootstrap CIs on ratios shown in executive reports, stability matters more than compute time.

[ADVERSARIAL]

Q: How does MetricLens handle a metric where the denominator is zero for the entire baseline period?

Zero denominator for an entire period means the metric is undefined for baseline — this is semantically a 'new metric' scenario, not a data quality issue. The behavior: $\text{weight}=0$ for baseline, $\text{rate}=0$ for baseline. $\text{total_delta} = \text{current_rate} * \text{current_weight}$. The entire delta appears as cross_term after zero-filling. Interpretation: all growth is from the metric not existing before. The profiler's `null_rate` check will flag if this is unexpectedly zero.

[ADVERSARIAL]

Q: Can MetricLens tell me which segment I should fix?

No, and this is an important boundary. MetricLens identifies which segment has the largest total_effect — the statistical answer to 'what explains the movement.' Whether to act on that segment depends on: (1) Is the effect statistically significant? (2) Is it actionable? (3) Are there confounders? MetricLens answers question 1 partially via bootstrap CIs, but 2 and 3 require domain expertise. The mandatory interpretation note in every output is a structural reminder of this boundary.

[ADVERSARIAL]

Q: What happens if your date column has mixed timezones?

Mixed timezone strings (e.g., '2024-01-01T08:00:00+05:30' vs '2024-01-01T10:00:00+00:00') that pandas parses as tz-aware but inconsistent will raise a `TypeError` on comparison. MetricLens expects naive datetime or

consistent tz-aware. Preprocessing requirement: normalize to UTC before passing. The profiler does not currently check for timezone inconsistency — this is a known gap for a future quality check.

[ADVERSARIAL]

Q: Your Shapley formula assumes two players. What if you have three factors: mix, rate, AND seasonality?

Three-player Shapley requires evaluating all $3! = 6$ orderings and averaging. MetricLens currently implements two-player Shapley only (mix vs. rate). Seasonality is not a decomposition dimension in the current model — it would need to be captured as a confounding segment dimension (e.g., day-of-week as a dimension column). True three-player Shapley is a research-grade extension requiring a causal model to define the seasonality player's contribution function.

[ADVERSARIAL]

Q: How does your bootstrap CI interact with zero-filled segments?

Zero-filled segments (new or disappeared) have rate=0 in one period. During bootstrap resampling of date-level aggregates, their contribution is constant across resamples — no variance. This means the CI for a zero-filled segment is exactly [0, 0] for that period's contribution. The CI correctly reflects zero uncertainty about a segment that contributed nothing. This is statistically correct, not a bug.

[ADVERSARIAL]

Q: MetricLens doesn't test for statistical significance between periods — isn't that a major gap?

Yes, explicitly acknowledged. The correct workflow is: MetricLens → identify top segments → t-test or Mann-Whitney U on segment-level rates between periods. MetricLens bootstrap CIs give a distribution of the decomposition coefficient, not a p-value for the rate difference. They are complementary: CIs tell you the precision of the decomposition; significance tests tell you whether the rate change is larger than sampling noise. Future integration: run significance test on top_k segments automatically.

[PRODUCTION]

Q: How would you test this in a production pipeline with 50 metrics running nightly?

Three test layers: (1) Unit — test decomposition identity holds for synthetic data with known segments. (2) Integration — run full analyze() on fixture DataFrames, assert top_segment matches expected. (3) Property — pytest-hypothesis generates random segment weights/rates, asserts $\text{abs}(\text{sum}(\text{mix}+\text{rate}+\text{cross}) - \text{total_delta}) < 1e-10$. Production monitoring: emit decomposition result to metrics warehouse, alert when identity violation $> 1e-8$ (hardware floating-point anomaly detector).

[PRODUCTION]

Q: How do you version the decomposition output for audit trails?

AnalysisResult includes: metric_name, baseline_period, current_period, run_timestamp, git_sha (injected at build time via importlib.metadata or environment variable). Store JSON output to append-only store keyed by (metric_name, date_range, sha). Downstream dashboards reference the versioned result, not a live recomputation. Re-running with same inputs produces bit-identical output — decomposition is deterministic.

[PRODUCTION]

Q: What's your strategy for handling schema drift in the input DataFrame?

DataProfiler checks column presence and type at analyze() time. Schema drift surfaces as: FAIL if required columns missing, WARN if dtypes differ from expected. In production: wrap analyze() in a try/except that catches DataProfiler FAIL results and routes to a dead-letter queue for manual inspection. Never silently proceed with degraded inputs — the output looks valid but reflects wrong data.

[PRODUCTION]

Q: How would you deploy MetricLens behind a REST API?

FastAPI endpoint: POST /analyze with body {metric_spec, baseline_period, current_period, data_ref}. data_ref is a BigQuery query string — executed server-side. Response: AnalysisResult JSON. Caching: Redis keyed by hash(metric_spec + periods + query). TTL: 24h for nightly data. Rate limiting: 100 req/min per API key. Auth: API key header validated against secrets manager. Latency target: <2s for <10K segment-day rows (BQ query dominates).

[PRODUCTION]

Q: What monitoring would you add to a production MetricLens deployment?

Four signal classes: (1) Data quality — profiler FAIL/WARN rate per metric over time. Rising FAIL rate = upstream pipeline degradation. (2) Decomposition stability — track top_segment change rate. If top segment changes every day, either the metric is genuinely volatile or data quality issues are causing noise. (3) Latency — P99 analyze() latency alert if >5s. (4) Identity check — abs(sum - total_delta) metric, alert if > 1e-8 (should never fire; indicates numerical library regression).

[CROSS-PROJECT]

Q: How would MetricLens integrate with NexusSupply?

NexusSupply tracks supplier financial health scores over time. MetricLens could decompose month-over-month changes in average health score by supplier_tier (large/medium/small) and region. Example: 'Average supplier health dropped 0.08 — 60% explained by mix shift toward small suppliers, 40% by rate decline in APAC tier.' This is a concrete use case where MetricLens adds interpretability to NexusSupply's aggregate metrics.

[CROSS-PROJECT]

Q: How would MetricLens integrate with InferenceLens?

InferenceLens tracks routing quality and cost per task type over time. MetricLens could decompose week-over-week changes in average quality score by model_tier and task_type. Example: 'Quality dropped 0.04 — 70% explained by mix shift toward local-tier routing (cost optimization), 30% by rate decline in summarization tasks.' MetricLens makes InferenceLens routing trade-offs interpretable to business stakeholders.

[CROSS-PROJECT]

Q: If you were building a unified analytics platform, where does MetricLens fit?

MetricLens is the metric interpretation layer: sits between data warehouse (raw events) and dashboards (human-readable insights). Pipeline: BigQuery → segment aggregates → MetricLens decomposition → metrics warehouse → dashboard. The MetricLens layer is stateless and idempotent — same inputs always produce same outputs. It is not a data store; it is a computation module.

[BEHAVIORAL]

Q: You found a bug in the decomposition 6 months after launch — how do you handle it?

Four-step response: (1) Characterize — when does the bug fire, how large is the error, what decisions were made using incorrect output. (2) Fix forward — patch and test. (3) Backfill — re-run all historical decompositions with corrected logic, version the new outputs, alert downstream consumers. (4) Document — RCA in changelog, add regression test that would have caught it. The identity property is the key test: if $\text{sum}(\text{mix} + \text{rate} + \text{cross}) \neq \text{total_delta}$, the decomposition is wrong.

[BEHAVIORAL]

Q: A PM says MetricLens is too complicated — executives want a single number. How do you respond?

The single number is `total_delta` — MetricLens gives you that plus the explanation. The executive dashboard shows: `total_delta`, `top_segment`, `top_segment_effect%`, and `direction` (mix-driven vs. rate-driven). Three numbers. The full decomposition is one click away for analysts who need to act on it. Complexity is in the engine, not the interface. MetricLens makes the default output simple; the detail is opt-in.

[BEHAVIORAL]

Q: How do you explain Shapley attribution to a non-technical stakeholder?

Imagine two people moved a rock together — how do you split credit? If person A went first and made it easier, person B's credit is inflated by A's setup. Shapley averages over both orderings: 'what would each person have contributed if they went first? If second?' The average is fair. In MetricLens: mix and rate both contributed to the metric change. The `cross_term` is their joint effect — Shapley splits it 50/50 because we have no reason to give more credit to either factor.

Section 14 — Worked Numerical Examples (Show Your Work)

Interviewers frequently ask you to walk through the math by hand. Memorize Example 1 (two-segment case) in full. Examples 2 and 3 cover edge cases.

Example 1: Two-Segment CVR Decomposition (Standard Case)

Setup: Metric = CVR (RatioMetric). Dimension = channel. Two segments: paid_search and organic. Baseline period: Jan 1-7. Current period: Jan 8-14.

Segment	B sessions	B orders	B weight	B rate	C sessions	C orders	C weight	C rate
paid_search	6,000	300	0.60	0.050	8,000	440	0.727	0.055
organic	4,000	200	0.40	0.050	3,000	165	0.273	0.055
TOTAL	10,000	500	1.00	0.050	11,000	605	1.00	0.055

Step 1 — $\text{total_delta} = 0.055 - 0.050 = 0.005$ (CVR improved by 0.5pp) Step 2 — paid_search: $\text{weight_delta} = +0.127$, $\text{mix_effect} = 0.127 \times 0.050 = +0.00635$, $\text{rate_effect} = 0.60 \times 0.005 = +0.003$, $\text{cross_term} = 0.127 \times 0.005 = +0.000635$ Step 3 — organic: $\text{weight_delta} = -0.127$, $\text{mix_effect} = -0.127 \times 0.050 = -0.00635$, $\text{rate_effect} = 0.40 \times 0.005 = +0.002$, $\text{cross_term} = -0.127 \times 0.005 = -0.000635$ Step 4 — $\text{sum mix} = -0.00635 + 0.00635 = 0.000$, $\text{sum rate} = 0.003 + 0.002 = 0.005$, $\text{sum cross} = 0.000$ Identity check: $0.000 + 0.005 + 0.000 = 0.005 = \text{total_delta}$ ✓ Interpretation: The CVR improvement is 100% rate-driven — both channels improved CVR by 0.5pp. Mix shifted toward paid_search (larger weight) but since rates moved identically, mix contributed nothing.

Example 2: Disappeared Segment (Segment Present in Baseline Only)

Setup: Same CVR metric. Add segment 'email'. Baseline: email has 2,000 sessions, 80 orders (rate=0.040). Current: email channel was shut down — zero sessions, zero orders.

Zero-fill: email appears in current with sessions=0, orders=0, weight=0.0, rate=0.0. $\text{weight_delta} = 0.0 - (2000/12000) = -0.167$ $\text{mix_effect} = -0.167 \times 0.040 = -0.00667$ $\text{rate_effect} = (2000/12000) \times (0.0 - 0.040) = 0.167 \times (-0.040) = -0.00667$ $\text{cross_term} = (-0.167) \times (-0.040) = +0.00667$ Sum for email: $-0.00667 - 0.00667 + 0.00667 = -0.00667$ Shapley(mix) = $-0.00667 + 0.00667/2 = -0.00334$ Shapley(rate) = $-0.00667 + 0.00667/2 = -0.00334$ Interpretation: Email closure equally penalized mix AND rate attribution — correct, since shutting a channel simultaneously removes its weight contribution and its rate contribution. The zero-fill and outer merge are what make this tractable without special-case code.

Example 3: Simpson's Paradox — When Top-Level Rate Improves Despite Both Segments Declining

Setup: CVR metric by device. Segments: desktop, mobile. Baseline: desktop 8,000 sessions 400 orders (rate=0.050), mobile 2,000 sessions 60 orders (rate=0.030). Total CVR = $460/10,000 = 0.046$. Current: desktop 3,000 sessions 135 orders (rate=0.045), mobile 7,000 sessions 196 orders (rate=0.028). Total CVR = $331/10,000 = 0.0331$. Observation: Both desktop rate (0.050→0.045, -0.005) and mobile rate (0.030→0.028, -0.002) DECLINED. Yet... total CVR = $0.0331 < 0.046$ → ALSO declined. Wait — no paradox here. Let us recalculate. The classic Simpson's Paradox case: Overall metric IMPROVES while all per-segment rates DECLINE — possible when composition shifts toward high-rate segments. For example if mobile had higher CVR and mix shifted toward mobile, aggregate CVR could rise even if both segment rates fell. MetricLens decomposition exposes this: total_delta would show positive total, negative $\text{sum}(\text{rate_effects})$, and a large positive $\text{sum}(\text{mix_effects})$. The analyst sees: 'Rate hurt us, mix saved us.' Without decomposition, an analyst sees only the positive aggregate and concludes rates improved — precisely the paradox that creates wrong business decisions.

Section 15 — Technical Architecture Reference

This section provides a dense reference of every architectural decision, file, constant, and interface in MetricLens. Use it as a lookup table when preparing for technical screen rounds.

Module Map: What Lives Where

Module	Key Functions	Purpose
decomposition.py	ratio_decomposition(), additive_decomposition(), _bootstrap_ratio_cis()	Core math: mix/rate/cross identity, Shapley, bootstrap CI, FLAT_EPSILON=1e-9
core.py	MetricLens class, analyze(), analyze_cross_dimensions()	Entry point: dispatches to metric type, calls DataProfiler, calls decomposition, builds AnalysisResult
metrics.py	MetricSpec, RatioMetric, SumMetric, CountMetric, AverageMetric	Metric type hierarchy: decomposition_type dispatch, aggregation logic, optional weight_col
profiler.py	DataProfiler, QualityCheckResult, profile()	10 quality checks: date coverage, null rate, duplicate grain, future dates, etc.
report.py	AnalysisResult, MANDATORY_INTERP RETATION_NOTE	Output schema: to_json(), to_markdown(), to_html(), schema_version, interpretation_note
exceptions.py	DecompositionTypeError, ZeroDenominatorError, ProfilerFailError	Structured exceptions for invalid metric type, zero denom, profiler FAIL blocking

Key Constants and Thresholds (Know by Heart)

Constant	Value	File	Meaning
FLAT_EPSILON	1e-9	decomposition.py	Relative threshold below which total_delta treated as zero
schema_version	'1.0'	report.py	Version tag on every AnalysisResult JSON output
date_coverage FAIL	>20%	profiler.py	Missing dates fraction triggering FAIL verdict
null_rate WARN	>3%	profiler.py	Null fraction in dimension/numerator/denominator column
null_rate FAIL	>15%	profiler.py	Null fraction triggering FAIL verdict
bootstrap_n	10,000	decomposition.py	Number of date-level bootstrap resamples
ci_level	0.95	decomposition.py	Default confidence interval level (95%)
default_seed	42	decomposition.py	numpy.random.default_rng seed for reproducibility
cross_term formula	$(w_c - w_b) * (r_c - r_b)$	decomposition.py	Per-segment cross-term in ratio decomposition
Shapley mix	mix + cross/2	decomposition.py	Mix player Shapley value in 2-player game

Data Flow: analyze() Call Sequence

1. analyze(df, metric_spec, baseline_period, current_period, dim_cols) called. 2. DataProfiler.profile(df, baseline_period, current_period, dim_cols) — 10 checks. FAIL check → raises ProfilerFailError (if strict=True). 3.

Filter df to baseline rows. GROUP BY dim_cols. Compute segment numerator, denominator, weight, rate. 4. Filter df to current rows. Same aggregation. 5. outer merge on dim_cols, fillna(0.0) — zero-fills disappeared/new segments. 6. Dispatch: RatioMetric/AverageMetric → ratio_decomposition(); SumMetric/CountMetric → additive_decomposition(). 7. In ratio_decomposition(): compute mix_effect, rate_effect, cross_term per segment. Apply Shapley. If bootstrap=True: call _bootstrap_ratio_cis(). Sort by abs(total_effect) descending. 8. Compute total_delta, verify identity: assert abs(sum(effects) - total_delta) < 1e-10. 9. Build AnalysisResult with all fields, attach MANDATORY_INTERPRETATION_NOTE. 10. Return AnalysisResult. Caller accesses .to_json(), .to_markdown(), .to_html().

AnalysisResult Schema (16 Key Fields)

Field	Type	Description
metric_name	str	Name of the metric analyzed
decomposition_type	str	'ratio' or 'additive'
baseline_period	tuple[str,str]	Start/end date strings for baseline
current_period	tuple[str,str]	Start/end date strings for current
total_delta	float	Current rate minus baseline rate (exact)
direction	str	'INCREASE'/'DECREASE'/'FLAT'
segments	list[SegmentResult]	Per-segment decomposition, sorted by abs(total_effect)
top_segment	str	Segment name with largest abs(total_effect)
mix_dominated	bool	True if abs(sum mix_effects) > abs(sum rate_effects)
rate_dominated	bool	True if abs(sum rate_effects) > abs(sum mix_effects)
quality_checks	list[QualityCheckResult]	10 profiler results with status/message
bootstrap_cis	dict None	Nested dict of {lo,hi,mean} per segment per field, or None
interpretation_note	str	MANDATORY_INTERPRETATION_NOTE constant text
run_timestamp	str	ISO-8601 UTC timestamp of analyze() call
schema_version	str	'1.0'
dim_cols	list[str]	Dimension columns used in this analysis

Section 16 — Production Readiness Checklist

This checklist represents the gap between a research prototype and a production deployment. Know which items are implemented (green), which are partial (yellow), and which are roadmap (red). Interviewers respect candidates who can articulate this boundary precisely.

Status	Feature	Notes
IMPLEMENTED	Algebraically exact decomposition identity	Zero residual, mathematically proven
IMPLEMENTED	Shapley attribution for cross_term	Ordering-bias-free, 2-player exact
IMPLEMENTED	Date-level bootstrap CI	10K resamples, seed=42, reproducible
IMPLEMENTED	10-check DataProfiler	date_coverage, null_rate, duplicate_grain, future_dates, etc.
IMPLEMENTED	Zero-fill via outer merge	Disappeared/new segments handled without special cases
IMPLEMENTED	FLAT_EPSILON guard	1e-9 relative threshold prevents noise decompositions
IMPLEMENTED	Structured JSON/Markdown/HTML output	schema_version='1.0', interpretation note
IMPLEMENTED	Four metric types with correct dispatch	RatioMetric, AverageMetric, SumMetric, CountMetric
IMPLEMENTED	Full test suite	38+ tests covering identity, Shapley, bootstrap, profiler
PARTIAL	Cross-dimension analysis	Implemented for 2D; >2D requires Cartesian join, not supported
PARTIAL	Error handling	Main exceptions defined; edge cases (schema drift) need more coverage
PARTIAL	Performance benchmarks	O(S) analysis documented; no benchmark suite with SLA targets
ROADMAP	Statistical significance testing	Identify top segments but not whether effect > noise
ROADMAP	Minimum segment size filter	min_segment_size param to aggregate sparse segments
ROADMAP	Timezone validation	No check for mixed tz-aware vs. naive datetime columns
ROADMAP	Overlapping period validation	No check preventing baseline and current from overlapping
ROADMAP	REST API wrapper	FastAPI endpoint not yet implemented
ROADMAP	Causal inference integration	DoWhy/CausalML integration not yet implemented
ROADMAP	Real-time streaming mode	Requires micro-batch accumulator; not in current architecture
ROADMAP	Multi-metric correlation	No correlation analysis across multiple MetricLens outputs

How to use this checklist in an interview: When asked 'is this production-ready?' do NOT say yes or no. Say: 'The core computation is production-ready — algebraically exact, tested, structured output. What it lacks for a full production deployment is: statistical significance pairing, minimum segment size filtering, and a REST API wrapper. Those are 3-5 days of engineering. The math and data quality layers are already production-grade.'

Appendix A — Annotated Source Code Reference

This appendix walks through the most interview-critical code paths in MetricLens. Being able to trace from 'what does it do' to 'where is that in the code' is the difference between a candidate who built the project and one who read the README.

A.1 `ratio_decomposition()` — The Core Math (`decomposition.py`)

This is the most important function in MetricLens. Every interview defense flows through here.

```
def ratio_decomposition(metric, baseline_df, current_df, dimension,
                        baseline_value, current_value) -> list[dict]:
    # Step 1: Compute per-segment stats for each period
    baseline = metric.compute_by_group(baseline_df, dimension)
    # Returns: DataFrame with [segment, numerator, denominator, rate]
    # rate = numerator/denominator per segment; 0 if denominator=0

    # Step 2: Outer merge – zero-fills disappeared/new segments
    merged = baseline.merge(current, on='segment', how='outer').fillna(0.0)
    # CRITICAL: 'outer' not 'inner'. 'fillna(0.0)' is the zero-fill.
    # Without this, dropped segments break the identity sum.

    # Step 3: Compute weights (share of denominator in each period)
    merged['baseline_weight'] = merged['baseline_denominator'] / baseline_denominator_total
    merged['current_weight'] = merged['current_denominator'] / current_denominator_total
    # Weights sum to 1.0 in each period by construction (after zero-fill)

    # Step 4: The three decomposition terms
    merged['mix_effect'] = merged['weight_delta'] * merged['baseline_rate']
    merged['rate_effect'] = merged['baseline_weight'] * merged['rate_delta']
    merged['cross_term'] = merged['weight_delta'] * merged['rate_delta']
    merged['total_effect'] = merged['mix_effect'] + merged['rate_effect'] + merged['cross_term']

    # Step 5: Shapley attribution – 50/50 split of cross_term
    merged['mix_attributed'] = merged['mix_effect'] + merged['cross_term'] / 2
    merged['rate_attributed'] = merged['rate_effect'] + merged['cross_term'] / 2
    # Comment in code: 'Shapley value for 2-effect cooperative game'
    # mix_attributed + rate_attributed = total_effect exactly

    # Step 6: Sort by abs(total_effect) – investigation priority
    merged = merged.sort_values('total_effect', key=lambda s: s.abs(), ascending=False)
```

Key interview lines to know: the 'outer' merge on line 2, the two formulas for weight on line 3, the three decomposition assignments on lines 4-6, and the Shapley split on lines 7-8. If an interviewer asks 'show me the code', these are the 8 lines to point at.

A.2 `_bootstrap_ratio_cis()` — Date-Level Resampling (`core.py`)

The bootstrap function is critical for any question about uncertainty quantification. Key implementation details to know:

```
rng = np.random.default_rng(seed) # reproducible – seed=42 by default
baseline_dates = baseline_df_full[date_col].unique() # array of date values
current_dates = current_df_full[date_col].unique()

for _ in range(n_bootstrap):
    # Resample DATES (not rows) with replacement
```

```

b_dates = rng.choice(baseline_dates, size=len(baseline_dates), replace=True)
c_dates = rng.choice(current_dates, size=len(current_dates), replace=True)
# Keep ALL rows for each sampled date – preserves within-day correlation
b_sample = baseline_df_full[baseline_df_full[date_col].isin(b_dates)]
c_sample = current_df_full[current_df_full[date_col].isin(c_dates)]
# Re-run full decomposition on the resampled data
rows = ratio_decomposition(metric, b_sample, c_sample, dim, b_val, c_val)
# Accumulate field values per segment
for row in rows:
    seg = str(row['segment'])
    for f in ['mix_effect', 'rate_effect', 'cross_term', 'total_effect', ...]:
        accumulators[dim][seg][f].append(float(row.get(f)))
# After n_bootstrap iterations – compute percentile CIs
result[dim][seg][f] = {
    'mean': float(np.mean(arr)),
    'lo': float(np.quantile(arr, alpha)), # default: 0.025
    'hi': float(np.quantile(arr, 1-alpha)), # default: 0.975
}

```

The key line: `rng.choice(baseline_dates, size=len(baseline_dates), replace=True)`. This resamples the date ARRAY with replacement. A 7-day baseline with dates [Mon, Tue, Wed, Thu, Fri, Sat, Sun] might resample to [Mon, Mon, Wed, Thu, Thu, Sat, Sun] — two Mondays, two Thursdays. All rows from those sampled dates are included, preserving within-day correlation.

A.3 DataProfiler — 10 Quality Checks (profiler.py)

The 10 checks break down as: 2 row_count + 2 date_coverage + 1 period_length + 4 null_rate (one per dimension in the demo) + 1 duplicate_grain. Key thresholds to know:

```

# date_coverage: status logic
if missing_ratio == 0: status = 'PASS'
elif missing_ratio > 0.20: status = 'FAIL' # >20% dates missing → FAIL
else: status = 'WARN' # 1-20% missing → WARN

# null_rate_{dimension}: status logic
if rate >= 0.15: status = 'FAIL' # >=15% nulls → FAIL
elif rate >= 0.03: status = 'WARN' # 3-15% nulls → WARN
else: status = 'PASS'

# duplicate_grain: status logic
if duplicate_count == 0: status = 'PASS'
else: status = 'WARN' # any duplicate grain rows → WARN

```

Why are these thresholds these specific values? date_coverage 20% failure: losing more than 1 day of 5 means the period is not representative. null_rate 15% failure: more than 1 in 7 rows having null dimensions means a significant portion of the data is unclassifiable. null_rate 3% warning: 3% is the natural noise threshold for data quality issues that are worth flagging but not necessarily blocking.

A.4 MANDATORY_INTERPRETATION_NOTE — report.py

```

MANDATORY_INTERPRETATION_NOTE = (
    'MetricLens reports deterministic metric movement decomposition. '
    'It identifies segment contributors, mix effects, rate effects, and cross terms. '
    'It does not claim causality, statistical significance, anomaly detection, '

```

```
'or root cause proof. Use these outputs as investigation signals, not automatic decisions.'  
)
```

This string is assigned in `report.py` at module level and referenced in `core.py`'s `analyze()` method as: `payload['interpretation_note'] = MANDATORY_INTERPRETATION_NOTE`. It is a module-level constant, not configurable. It cannot be `None`. It cannot be overridden by an `AuditConfig`. Every output format — JSON, Markdown, HTML — includes it.

Appendix B — Mock Interview Simulations

Three simulated interview scenarios at increasing difficulty. Practice these verbally until the answers flow naturally. The goal is not to memorize words — it is to internalize the reasoning so you can reconstruct the answer from first principles under pressure.

B.1 Phone Screen (30 minutes) — L4 Data Engineer Role

Interviewer: Tell me about a project where you worked with metric analysis.

Candidate: I built MetricLens, a Python library for metric movement decomposition. The core problem it solves: when a business metric drops — say, conversion rate fell 8% week-over-week — every data team needs to answer 'which segments drove that, and was it because of changes in segment composition or changes in segment performance?' I packaged the math for that analysis as a pip-installable library. The key design decision was making the decomposition algebraically exact with zero residual — the mix effects, rate effects, and cross terms always sum precisely to the total delta, no unexplained remainder.

Interviewer: What's the hardest technical challenge you solved in this project?

Candidate: Handling disappeared segments correctly. When a segment exists in the baseline but has zero volume in the current period, a naive implementation just drops it from the groupby result. But that breaks the mathematical identity: the remaining segments' weights no longer sum to 1.0, so the decomposition leaves a residual that looks like an error. The fix is zero-filling — treating disappeared segments as having weight 0 and rate 0 in the current period. That produces a negative mix_effect for the disappeared segment, correctly representing that it contributed nothing in the current period. Zero-filling is the only approach that preserves the exact identity.

Interviewer: How would you scale this to handle 100 million daily events?

Candidate: MetricLens operates on segment-day aggregates, not on raw events. For 100 million daily events across 7 days, 4 dimensions, 50 segment values each, the aggregate DataFrame has about 3,000 rows — identical size regardless of event volume. The scaling bottleneck is the GROUP BY aggregation step, which happens before MetricLens even runs. I'd move that aggregation to BigQuery or Snowflake — run a SQL GROUP BY in the warehouse, pass the 3,000-row result DataFrame to MetricLens. The library code itself doesn't need to change.

B.2 Technical Deep-Dive (60 minutes) — L5 ML Engineer Role

This interview tests whether you understand the math, not just the output.

Interviewer: Prove to me that $\text{mix} + \text{rate} + \text{cross} = \text{total_delta}$. Walk me through the algebra.

Candidate: Sure. For segment i , define w_{ib} , w_{ic} as the baseline and current weights (share of total denominator), and r_{ib} , r_{ic} as the baseline and current rates. The metric is $R = \sum_i (w_i * r_i)$. $\text{mix_effect}_i = (w_{ic} - w_{ib}) * r_{ib}$. $\text{rate_effect}_i = w_{ib} * (r_{ic} - r_{ib})$. $\text{cross_term}_i = (w_{ic} - w_{ib}) * (r_{ic} - r_{ib})$. Sum all three for segment i : $w_{ic} * r_{ib} - w_{ib} * r_{ib} + w_{ib} * r_{ic} - w_{ib} * r_{ib} + w_{ic} * r_{ic} - w_{ic} * r_{ib} - w_{ib} * r_{ic} + w_{ib} * r_{ib} = w_{ic} * r_{ic} - w_{ib} * r_{ib}$. Sum over all segments: $\sum (w_{ic} * r_{ic}) - \sum (w_{ib} * r_{ib}) = R_{\text{current}} - R_{\text{baseline}} = \text{total_delta}$. The cancellation is exact. It works for any number of segments, assuming weights sum to 1.0 in each period — which zero-filling enforces.

Interviewer: Why split the cross_term 50/50 rather than attributing it to one component?

Candidate: Because attributing to either component introduces an ordering bias. The standard decomposition computes mix_effect at baseline_rate and rate_effect at baseline_weight. If you reversed the order — compute mix at current_rate, rate at current_weight — you'd get different numbers. Both orderings are mathematically valid. The question is which ordering is 'correct', and there's no principled answer without game theory. Shapley

values solve exactly this problem: the Shapley value for player i is their average marginal contribution across all orderings. With two players, that's $\text{mix_attributed} = \text{mix_effect} + \text{cross_term}/2$ and $\text{rate_attributed} = \text{rate_effect} + \text{cross_term}/2$. This satisfies three fairness axioms: efficiency (values sum to total), symmetry (equal contributors get equal values), and null player (non-contributing player gets zero). MetricLens reports both the raw and Shapley-attributed values so callers can choose.

Interviewer: Your bootstrap CI uses date-level resampling. Why not just use a parametric test?

Candidate: A parametric test would require distributional assumptions — typically normality — about the segment-level effects. But daily metric data often violates normality: there are day-of-week effects, promotional spikes, and occasionally extreme outlier days. Bootstrap CIs are non-parametric: they make no distributional assumptions and work for any shape of the underlying distribution. The date-level choice specifically is because sessions on the same day are not independent — they share day-level confounders like promotional emails, competitor actions, or weather. Row-level resampling would treat these correlated observations as independent, producing falsely narrow CIs. Date-level resampling treats the day as the natural unit of independence.

B.3 System Design (45 minutes) — Staff Engineer Role

Design MetricLens as a production service at a company with 50 tracked metrics, daily refreshes, 4 dimensions per metric, and 10 analysts reading results.

Interviewer: Walk me through the end-to-end architecture you'd build.

Candidate: I'd split it into three layers. First, the data layer: a BigQuery or Snowflake table called 'metrics_daily_aggregated' at (date, metric_name, dimension, segment) grain, populated by a nightly Spark or dbt job from the raw event log. Second, the compute layer: an Airflow DAG that runs after the aggregation job completes — for each of the 50 metrics, it calls MetricLens.analyze() with the last 7 days as current period and the prior 7 days as baseline, writes the JSON output to a 'decomposition_results' table partitioned by (metric_name, run_date). Third, the delivery layer: a FastAPI service reads from decomposition_results and serves the JSON to a Streamlit or Looker dashboard. Each analyst sees a waterfall chart per metric with segment drill-down. Alerts fire when $\text{abs}(\text{total_delta}) > \text{threshold}$ AND top_segment explains more than 50% of the delta.

Interviewer: What's your data model for storing decomposition results?

Candidate: I'd store two tables. First, 'decomposition_runs' with schema: run_id, metric_name, baseline_start, baseline_end, current_start, current_end, run_timestamp, baseline_value, current_value, total_delta, relative_delta_pct, direction, schema_version, package_version. Second, 'decomposition_segments' with schema: run_id, dimension, segment, mix_effect, rate_effect, cross_term, total_effect, mix_attributed, rate_attributed, contribution_pct, baseline_rate, current_rate, baseline_weight, current_weight. Primary key on (run_id, dimension, segment). This schema supports: time-series of decompositions per metric, segment drill-down queries, and diffing two runs to see how the decomposition changed. The JSON from MetricLens maps directly to these two tables with a simple parse step.

Interviewer: How would you handle metric definition changes over time?

Candidate: This is the hardest operational problem for a decomposition system. If the metric definition changes — say, CVR starts counting mobile checkouts that were previously excluded — historical decompositions become incomparable to new ones. My approach: version the metric definition. Every MetricLens run gets a metric_definition_version field in the run metadata. When a definition changes, bump the version. The dashboard shows a discontinuity marker at version boundaries. Analysts are warned when comparing runs across versions. Optionally, backfill historical data with the new definition for continuity. This is the same problem as schema versioning in data warehouses — the solution is explicit versioning and metadata, not pretending the change didn't happen.

Appendix C — Comparison Matrix vs Alternatives

Interviewers often ask 'why did you build this instead of using X?' This matrix prepares you to answer any comparison question defensively.

Alternative	Strengths	Why MetricLens is Different
pandas groupby + diff	10-30 lines, no deps	No zero-fill identity guarantee, no Shapley, no bootstrap CI, no quality checks, no output format, no interpretation note, no segment priority ranking. Re-implemented per analyst, per team, with inconsistent edge case handling.
Statsig metric attribution	Production SaaS, experimentation-integrated	Proprietary and opaque — math is not auditable. Requires SaaS account and API. Cannot be run offline, in CI, or embedded in a custom pipeline. No residual guarantee.
Amplitude anomaly detection	Automated alerts, dashboard-integrated	Not a decomposition tool — detects anomalies but does not decompose them into mix/rate/cross. Attribution is heuristic, not algebraically exact. No zero residual.
Causal impact / synthetic control	Causal inference, intervention detection	Different question: causal inference answers 'what would have happened without the intervention?' MetricLens answers 'what drove the metric movement?' Causal methods require counterfactual construction; MetricLens is purely descriptive.
sklearn regression attribution	Model coefficients as attribution	Model-dependent and approximate. Assumes linearity. Coefficients change with model specification. Not algebraically exact. Cannot guarantee zero residual. Requires fitting, which adds variance and overfitting risk.
Google / Meta internal tools	Scale-optimized, infra-integrated	Not publicly available. MetricLens is open-source, pip-installable, and framework-agnostic. Serves the same analytical need for teams without access to proprietary internal tooling.

C.2 When NOT to use MetricLens

Intellectual honesty in interviews means knowing when your tool is the wrong choice:

- **When you need causal attribution:** MetricLens decomposes historical movement. If you need to know 'did the marketing campaign CAUSE the CVR lift?' you need a causal inference framework (DiD, synthetic control, regression discontinuity).
- **When you need real-time anomaly detection:** MetricLens requires complete baseline and current DataFrames. For real-time streaming anomaly detection, use a purpose-built time-series anomaly tool.
- **When your metric is not ratio/sum/count/average:** Exotic metric types (geometric mean, median, percentile, log-normal aggregates) do not fit the current decomposition framework without custom extension.
- **When statistical significance is the primary question:** MetricLens produces decomposition values, not significance tests. Use a stats testing framework as the primary tool and MetricLens as the investigation layer.
- **When you have fewer than ~30 rows per segment:** Sparse segments produce high-variance rate effects. The decomposition is mathematically correct but practically misleading. Aggregate to higher-level segments first.

Appendix D — Quick-Reference Card

D.1 The 5 Things to Say in the First 90 Seconds

- MetricLens decomposes metric movement into $\text{mix_effect} + \text{rate_effect} + \text{cross_term} = \text{total_delta}$, **algebraically exact**.
- Mix effect: segment grew or shrank in volume. Rate effect: segment's own performance changed. Cross term: interaction — always reported.
- Key implementation: **outer merge + fillna(0)** for disappeared/new segments preserves the identity sum.
- **Shapley split** of cross_term (50/50) eliminates ordering bias. **Bootstrap CI** via date-level resampling quantifies sampling uncertainty.
- It does NOT produce causality, significance, or forecasts. It produces investigation priorities.

D.2 Numbers to Know Cold

Item	Value and Context
FLAT_EPSILON	1e-9 — Floating-point guard threshold
Demo dataset	756 rows — 4 dimensions, seed=42, 7+7 day periods
Test count	38 passing — pytest from repo root
Default bootstrap	n=200, 95% CI — <code>analyze(bootstrap_n=200)</code>
Install	<code>pip install metriclens</code> — pandas + numpy only
Output fields	16 top-level — <code>schema_version</code> through <code>interpretation_note</code>
Quality checks	10 per <code>analyze()</code> — 2 <code>row_count</code> + 2 <code>date_coverage</code> + 1 <code>period_length</code> + 4 <code>null_rate</code> + 1 <code>duplicate_grain</code>
date_coverage FAIL	>20% dates missing — WARN at >0%, FAIL at >20%
null_rate FAIL	>=15% nulls in dimension — WARN at 3%, FAIL at 15%
schema_version	'1.0' — For downstream consumer versioning

D.3 Identity Formula (Latex-style for whiteboard)

```
R_current - R_baseline = total_delta
= Σ_i [ mix_effect_i + rate_effect_i + cross_term_i ]

where:
mix_effect_i = (w_ic - w_ib) * r_ib
rate_effect_i = w_ib * (r_ic - r_ib)
cross_term_i = (w_ic - w_ib) * (r_ic - r_ib)
w_i = segment_i_denominator / total_denominator (weight)
r_i = segment_i_numerator / segment_i_denominator (rate)

Shapley:
mix_attributed_i = mix_effect_i + cross_term_i / 2
rate_attributed_i = rate_effect_i + cross_term_i / 2
mix_attributed_i + rate_attributed_i = total_effect_i ✓
```

D.4 The Interpretation Note (memorize first sentence)

"MetricLens reports deterministic metric movement decomposition. It identifies segment contributors, mix effects, rate effects, and cross terms. It does not claim causality, statistical significance, anomaly detection, or root cause proof. Use these outputs as investigation signals, not automatic decisions."

This note is MANDATORY — it is a module-level constant in report.py, always in every output, and cannot be suppressed by any configuration parameter.

D.5 What Every Answer Should Include

- **State the claim:** 'The mix_effect tells you...'
- **Give the formula or code reference:** 'Computed as $(w_{ic} - w_{ib}) * r_{ib}$ in decomposition.py'
- **Give a concrete example:** 'In the CVR demo, paid_search has mix_effect = +0.00380...'
- **State the limitation:** 'This tells us the WHAT, not the WHY — no causality is implied'
- **Name the design decision:** 'We chose to always report the cross_term rather than absorbing it because...'